

Bitwise Operations

You are implementing a simple **bit-packing** routine. You have:

```
unsigned int buffer = 0; int totalBits = 0;
```

You receive three values to pack sequentially into buffer starting from the least-significant end:

Value	Number of valid bits
val1 = 0x5	bits1 = 3
val2 = 0xF	bits2 = 4
val3 = 0x2	bits3 = 2

(a) Write C code to pack all three values into buffer (val1 at the bottom, val2 above it, val3 above that).

C Pointers and Memory

Consider the following declarations:

```
char *words[4];
```

```
char buf[] = "hello world";
```

(a) What is the difference between `char *words[4]` and `char (*words)[4]`?

(b) After `words[0] = buf;`, what does `words[0][3]` evaluate to?

(c) Write code using `malloc` and `strcpy` to make `words[1]` point to its own independent copy of "goodbye".

argc / argv and String Handling

A program is invoked as:

```
./mygrep -i -n "hello world" data.txt
```

(a) What are the values of argc and each element of argv?

(b) Why is comparing an argument to a string using `if (argv[1] == "-i")` incorrect? Explain and show the correct way.

Process Control and wait

```
pid_t pid = fork();
if (pid == 0) {
    execlp("ls", "ls", "-l", NULL);
    perror("exec failed");
    exit(1);
}
int status;
wait(&status);
```

(a) Using the macros WIFEXITED, WEXITSTATUS, WIFSIGNALED, and WTERMSIG, write code that prints either the exit status or the signal number that terminated the child.

(b) Explain what happens if you call exec and it **fails**. Why is the perror / exit after execlp important?

(c) What happens if the parent never calls wait? What is the child process called in that state?

Unix I/O Redirection with System Calls

A shell needs to execute the command: `sort < input.txt > output.txt`

(a) List, **in order**, the system calls the shell makes (after fork, in the child) to set up the redirections and run sort. Be specific about arguments to open and dup2.

(b) Why must the original file descriptors from open be closed after calling dup2?

Pipes

A shell needs to execute: `cat file.txt | grep "error" | wc -l`

- (a)** How many calls to `pipe()` are needed? How many calls to `fork()`?
- (b)** Describe what each process does with the pipe file descriptors (which ends it dups and which it closes).
- (c)** What happens if the writing end of a pipe is never closed in the reading process? Why does this matter?

Signals

- (a)** Explain the difference between `signal()` and `sigaction()`. Give one specific capability that `sigaction()` provides that `signal()` does not.
- (b)** Write a short program that sets up a handler for `SIGALRM`, calls `alarm(5)`, then calls `pause()`. Explain what happens step by step.
- (c)** Can `SIGKILL` be caught or ignored? Explain why or why not from a system-design perspective.

Unix Commands and the Shell

- (a) Give a single shell pipeline that counts how many .c files exist anywhere under your home directory.
- (b) Explain the difference between Ctrl-C and Ctrl-Z in terms of which signal each sends and what the default behavior is.
- (c) You have a file myscript.sh. You type ./myscript.sh and get Permission denied. Show **two** different ways to still execute it.

make and Compilation

Given this Makefile:

```
myapp: main.o utils.o
    gcc -o myapp main.o utils.o

main.o: main.c defs.h
    gcc -c main.c

utils.o: utils.c defs.h
    gcc -c utils.c
```

- (a)** If you modify only defs.h, which targets get rebuilt and why?
- (b)** How does make decide whether a target needs to be rebuilt? Be specific about what it compares.
- (c)** If you type make and everything is up to date, then you run touch utils.c and type make again, what gets recompiled?

Threads and Synchronization

```
int counter = 0;
void *increment(void *arg) {
    for (int i = 0; i < 1000000; i++)
        counter++;
    return NULL;
}
```

(a) If two threads both run `increment`, will counter reliably equal 2,000,000 at the end? Explain why or why not.

(b) Fix the code using a **pthread mutex**. Show the declaration, initialization, and where lock/unlock calls go.

(c) Why do threads need locks but separate processes (with separate address spaces) typically do not?